

Posttraining

From language model to AI tool

Peter Schneider-Kamp, AI509, Fall 2026

Dishes of the Day

- ☐ Instruction tuning
- ☐ Parameter-efficient finetuning
- ☐ Preference alignment
- ☐ Process supervision



Instruction tuning

- pretrained LLMs are next word token predictors with context
- the context allows for in-context learning
- pretrained models perform surprisingly well on few-shot prompting
- however, most tasks require a diligently-trained instruction-tuned model

Supervised fine-tuning

- the model is trained with instructions / prompts / questions and maybe context
- examples of high-quality answers are provided
- loss calculation only considers the answers
- otherwise, technically the same as (continual) pre-training

Instruction following format

- each sample contains the fields "instruction", maybe "input", and "output"
- the "input" field contains context to reason about, e.g., a text to summarize
- Example in JSONL:

```
{"instruction": "What is JSONL?", "output": "A textual file format  
with one JSON value per row"}  
{"instruction": "Negate the sentence.", "input": "He was sad.",  
"output": "He was not sad."}
```

Multi-turn chat format 1/2 (ChatML)

- each sample contains a list of "messages"
- "messages" contain "role" and "content" fields
- "role" can be "system", "user", or "assistant"
- Example in JSONL:

```
{"messages": [{"role": "system", "content": "You are an unhelpful assistant."}, {"role": "user", "content": "Can you help me write a song?"}, {"role": "assistant", "content": "Why on God's green flat earth would I want to help you, dumbass?"}]}
```

Multi-turn chat format 2/2 (ShareGPT)

- each sample contains a list of "conversations" and maybe a "system" prompt
- "conversations" contain "from" and "value" fields
- "from" can be "human" or "gpt"
- Example in JSONL:

```
{"system": "You are an unhelpful assistant.", "conversations":  
[{"from": "human", "value": "Can you help me write a song?"},  
{"from": "gpt", "value": "Why on God's green flat earth would I  
want to help you, dumbass?"}]}
```

Model-dependent chat templates 1/2

- each model uses particular templates to convert instruction tuning data into pure text data
- Llama 3.x:

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>  
You are an unhelpful assistant.<|eot_id|>  
<|start_header_id|>user<|end_header_id|>  
Can you help me write a song?<|eot_id|>  
<|start_header_id|>assistant<|end_header_id|>  
Why on God's green flat earth would I want to help you, dumbass?  
<|eot_id|>
```

Model-dependent chat templates 2/2

- chat templates are often available through the `tokenizer.apply_chat_template` function
- typically implemented as Jinja templates:

```
{% if messages[0]['role'] == 'system' %}
    {% set offset = 1 %}
{% else %}
    {% set offset = 0 %}
{% endif %}
{{ bos_token }}
{% for message in messages %}
    {% if (message['role'] == 'user') != (loop.index0 % 2 == offset) %}
        {{ raise_exception('Conversation roles must alternate user/assistant/user/...') }}
    {% endif %}
    {{ '<|start_header_id|>' + message['role'] + '<|end_header_id|>\n\n' + message['content'] | trim +
    '<|eot_id|>' }}
{% endfor %}
{% if add_generation_prompt %}
    {{ '<|start_header_id|>' + 'assistant' + '<|end_header_id|>\n\n' }}
{% endif %}
```

Dishes of the Day

- ☒ Instruction tuning
- ☐ Parameter-efficient finetuning
- ☐ Preference alignment
- ☐ Process supervision



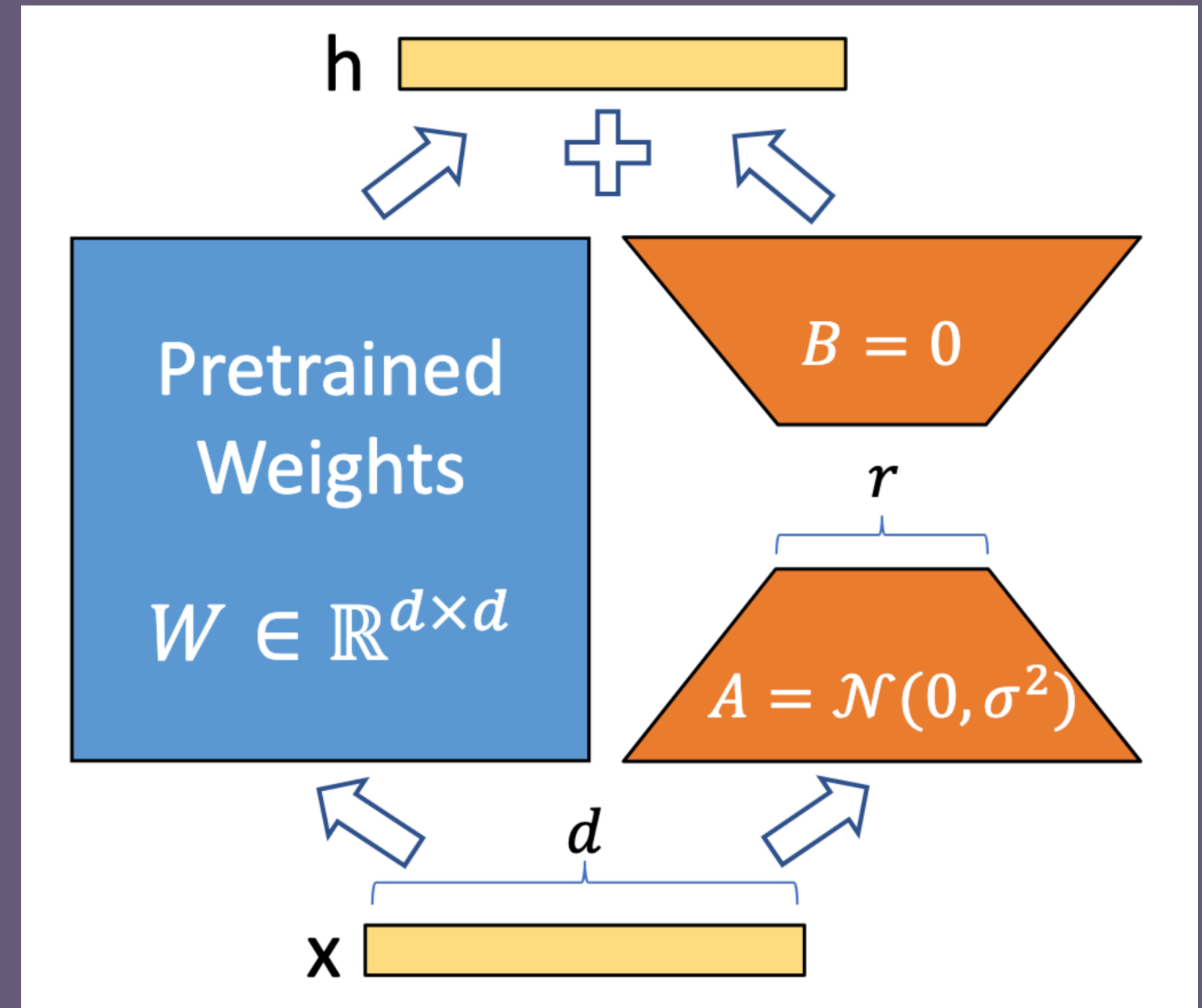
Parameter-efficient fine-tuning (PEFT)

Training overhead

- typical LLM training requires at least 6 bytes per parameter (16-bit weight, 2x 16-bit optimizer state)
- even training a 7B LLM needs 56 GByte GPU memory + activations & caches
- 'freezing' selected parameters reduces memory consumption to 2 bytes per weight

Low-rank adapters (LoRA)

- replace expensive $n \times m$ matrix L by $n \times r$ & $r \times m$ matrices A and B
- when $r \ll \min(n, m)$, we have that $|A| + |B| = nr + rm \ll nm = |L|$
- matrices A and B effectively cover the low ranks of L



from: <https://arxiv.org/abs/2106.09685>

Dishes of the Day

- ☒ Instruction tuning
- ☒ Parameter-efficient finetuning
- ☐ Preference alignment
- ☐ Process supervision



Preference alignment

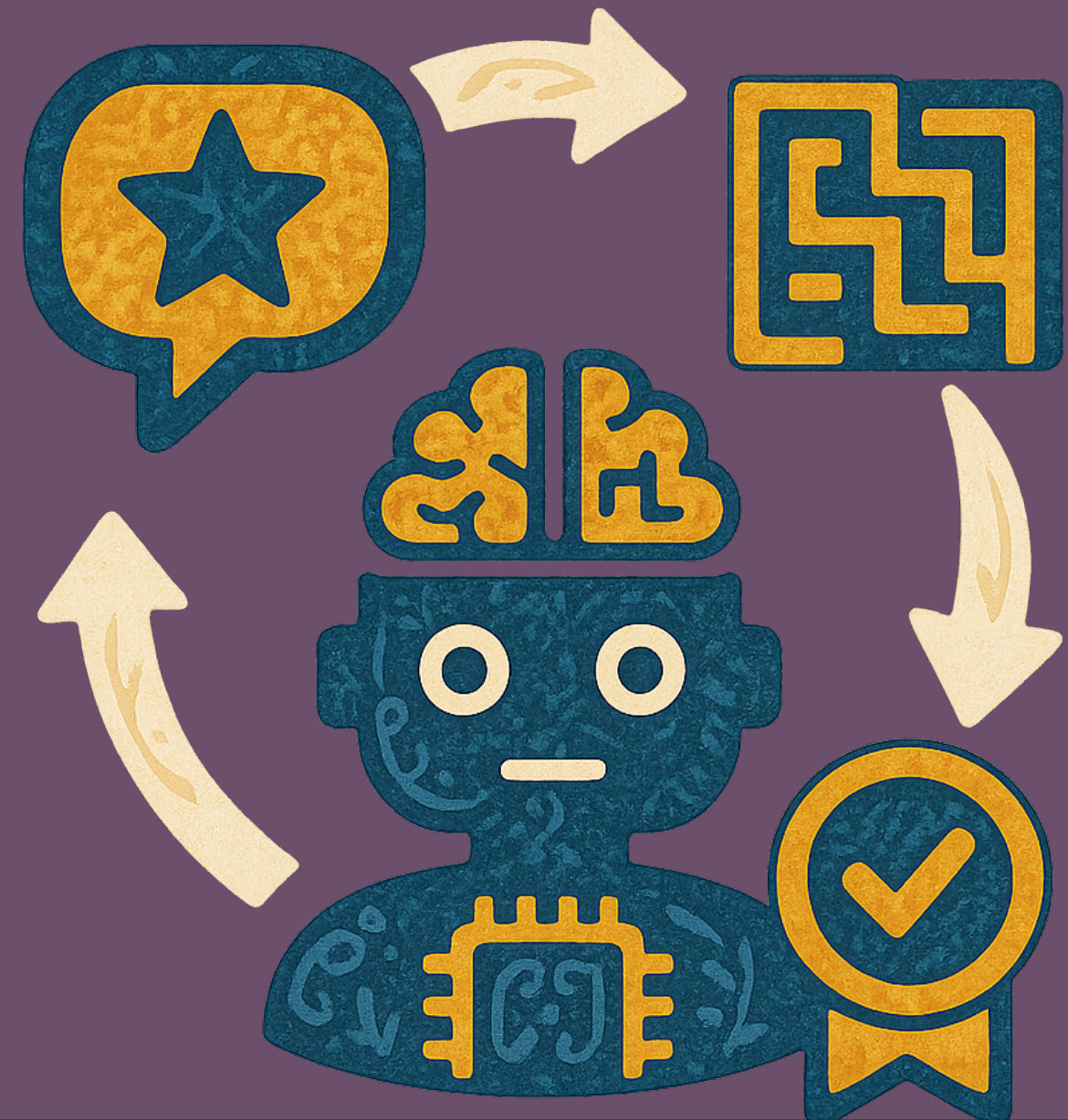
Reinforcement learning from human feedback (RLHF)

- generate different answers for the same prompt
- let humans rate these as positive (thumbs up) or negative (thumbs down)
- collect a dataset of "chosen" and "rejected" answers:

```
{"prompt": "Is Heineken better than Weihenstephan?", "chosen":  
[{"role": "assistant", "content": "Are you insane?"}],  
"rejected": [{"role": "assistant", "content": "Of course!"},  
{"role": "assistant", "content": "Yes, I love Heineken!"}]}
```

Reinforcement learning from AI feedback (RLAI)

- generate different answers for the same prompt
- let another language model rate these as positive or negative
- collect a dataset of "chosen" and "rejected" answers
- perform reinforcement learning using PPO (or GRPO)
- perform preference optimization using DPO



from: <https://medium.com/@oleglatypov/a-comprehensive-guide-to-proximal-policy-optimization-ppo-in-ai-82edab5db200>

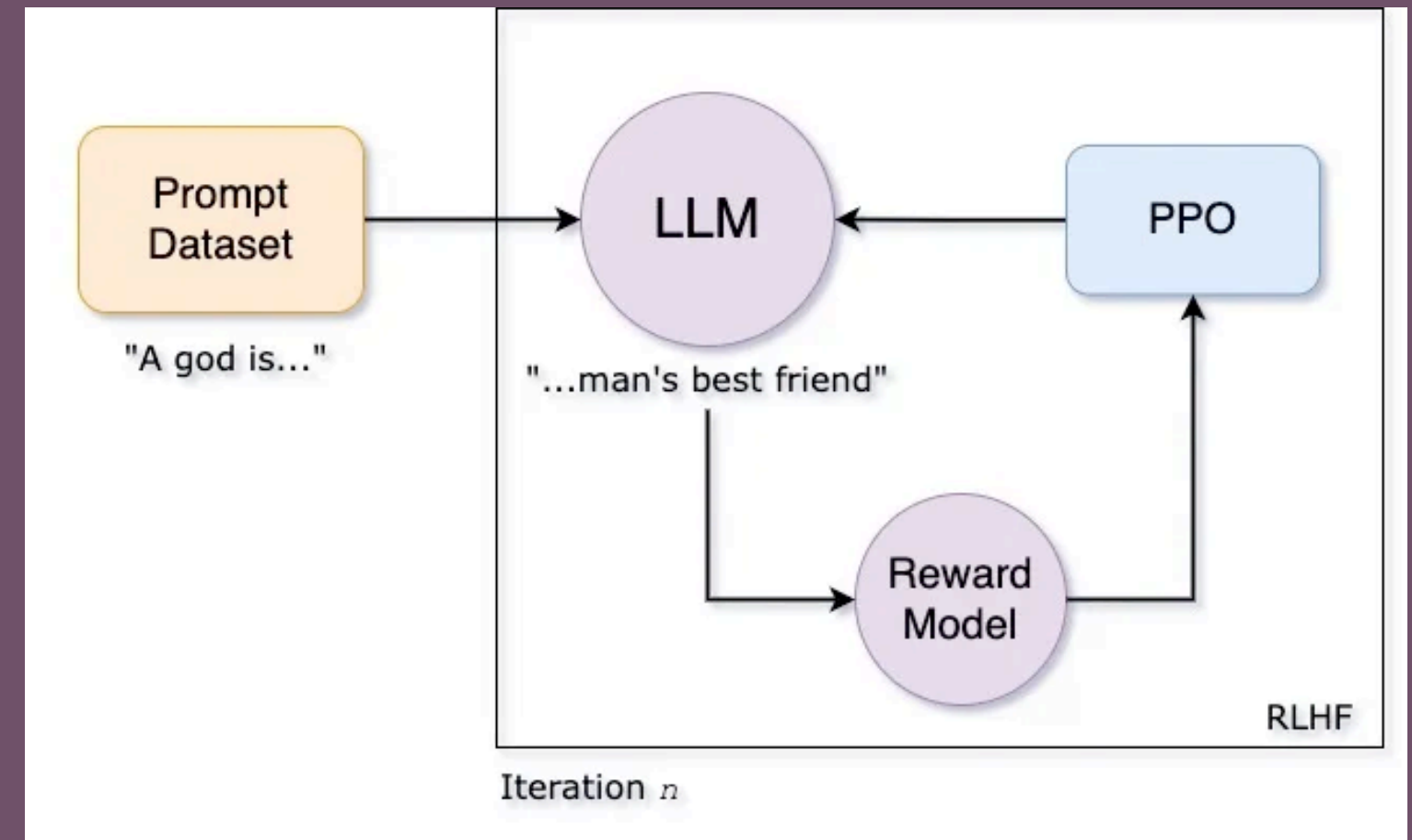
Proximal policy optimization (PPO)

- use the RLHF / RLAI dataset to train a reward model
- use the reward model to score answers produced by the LLM
- compute the advantage $\hat{A}_t$ as the difference between estimated reward and actual reward
- carefully adjust the LLM weights:

$$\mathcal{L}_{\text{policy}}(\theta) = \text{minClip}(r_t(\theta), \hat{A}_t, \epsilon)$$

where $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ and

$$\text{minClip}(r, A, \epsilon) = \min \left(r A, \text{clip} \left(r, 1 - \epsilon, 1 + \epsilon \right) A \right)$$

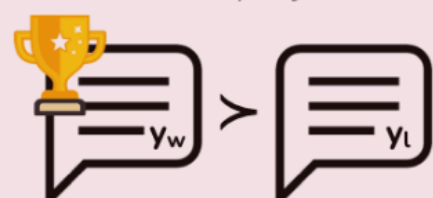


Direct preference optimization (DPO)

- get rid of the separate reward model
- get rid of reinforcement learning
- optimize the language model directly using KL divergence
- the language model is the implicit reward model

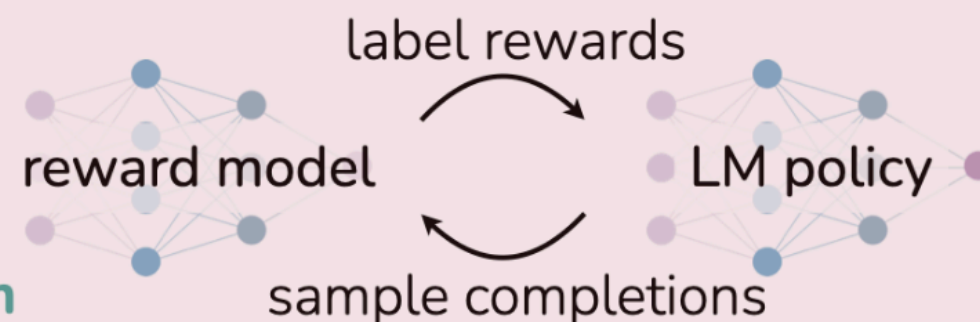
Reinforcement Learning from Human Feedback (RLHF)

x: "write me a poem about
the history of jazz"



preference data

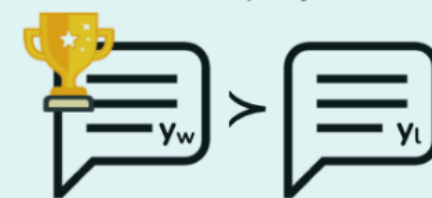
maximum
likelihood



reinforcement learning

Direct Preference Optimization (DPO)

x: "write me a poem about
the history of jazz"



preference data

maximum
likelihood



from: <https://arxiv.org/abs/2305.18290>

Group relative policy optimization (GRPO)

- score answers relatively in a group rather than separately
- advantage of an answer is how much it lies under or over the mean of the group:

$$A_i = \frac{r_i - \text{mean}(\{r_1, \dots, r_G\})}{\text{std}(\{r_1, \dots, r_G\})}$$

- loss function averages rewards within each group and stabilizes with respect to the reference:

$$\mathcal{L}_{\text{GRPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G \min \left(r_t(\theta) A_i, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) A_i \right)$$

$$- \beta D_{KL}(\pi_{\theta} || \pi_{ref})$$

Dishes of the Day

- ☒ Instruction tuning
- ☒ Parameter-efficient finetuning
- ☒ Preference alignment
- ☐ Process supervision



Process supervision

Test-time reasoning

- use extra time at inference to allow the model to plan, generate, evaluate, and finally present an answer
- the LLM generates tokens to this end and uses them as the context in the generation of the final answer
- the LLM might use external tools to add to this context (from web search to mathematical reasoning systems)
- the LLM might "reflect" on the contents of the context

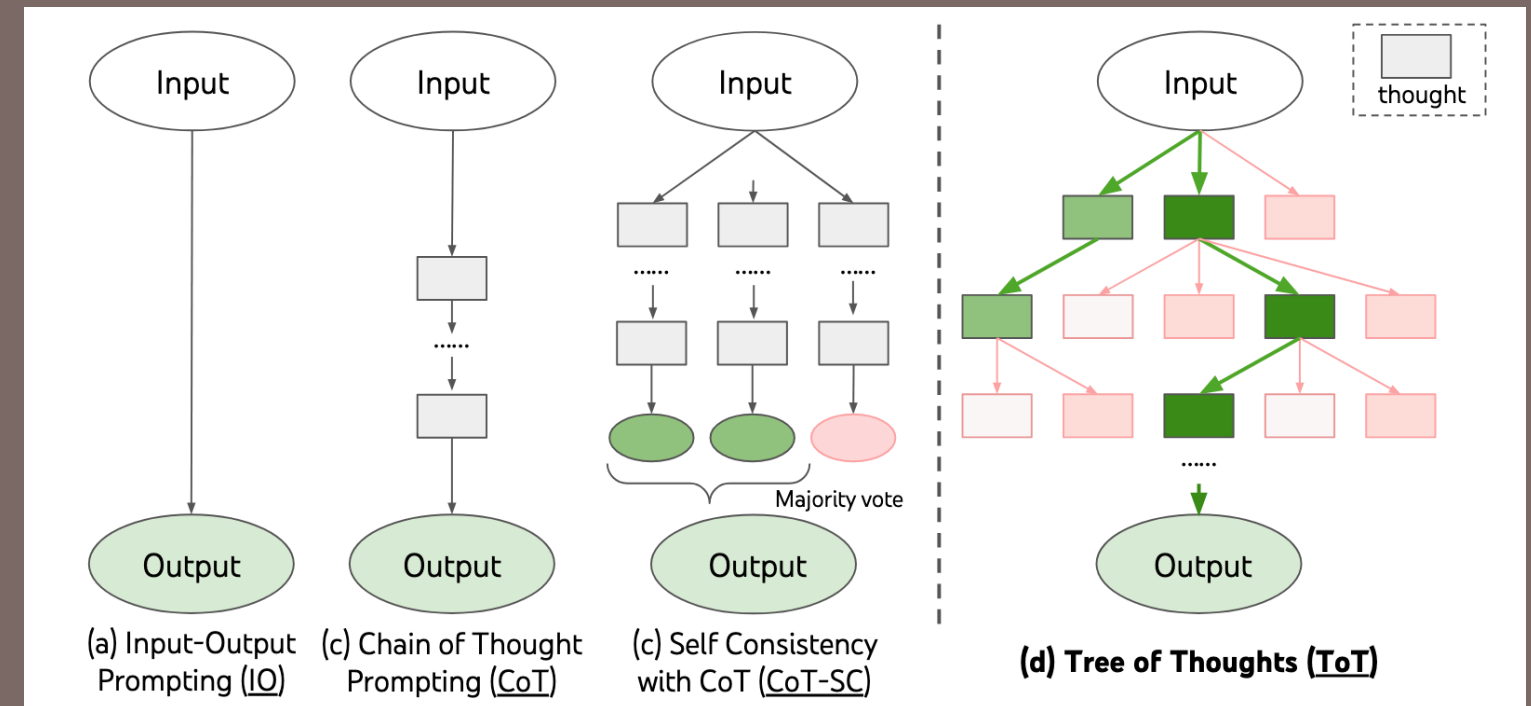
Chain-of-thought (CoT)

- reason linearly, i.e., step-by-step
- each step can draw on the previous steps as context
- Example of training data:

```
{"id": "ex-001", "task": "math_word_problem", "input": "Train A leaves at 1pm at 60 km/h, Train B leaves 30 min later at 80 km/h, same track. When does B catch A?", "rationale": ["Let t be hours after 1pm when B catches A.", "A distance = 60*t.", "B has 0.5h delay, so B time = t - 0.5 (must be > 0).", "B distance = 80*(t - 0.5).", "Set equal: 60t = 80(t - 0.5) -> 60t = 80t - 40 -> 20t = 40 -> t=2.", "Catch-up time = 1pm + 2h = 3pm."], "final_answer": "3:00 PM"}
```

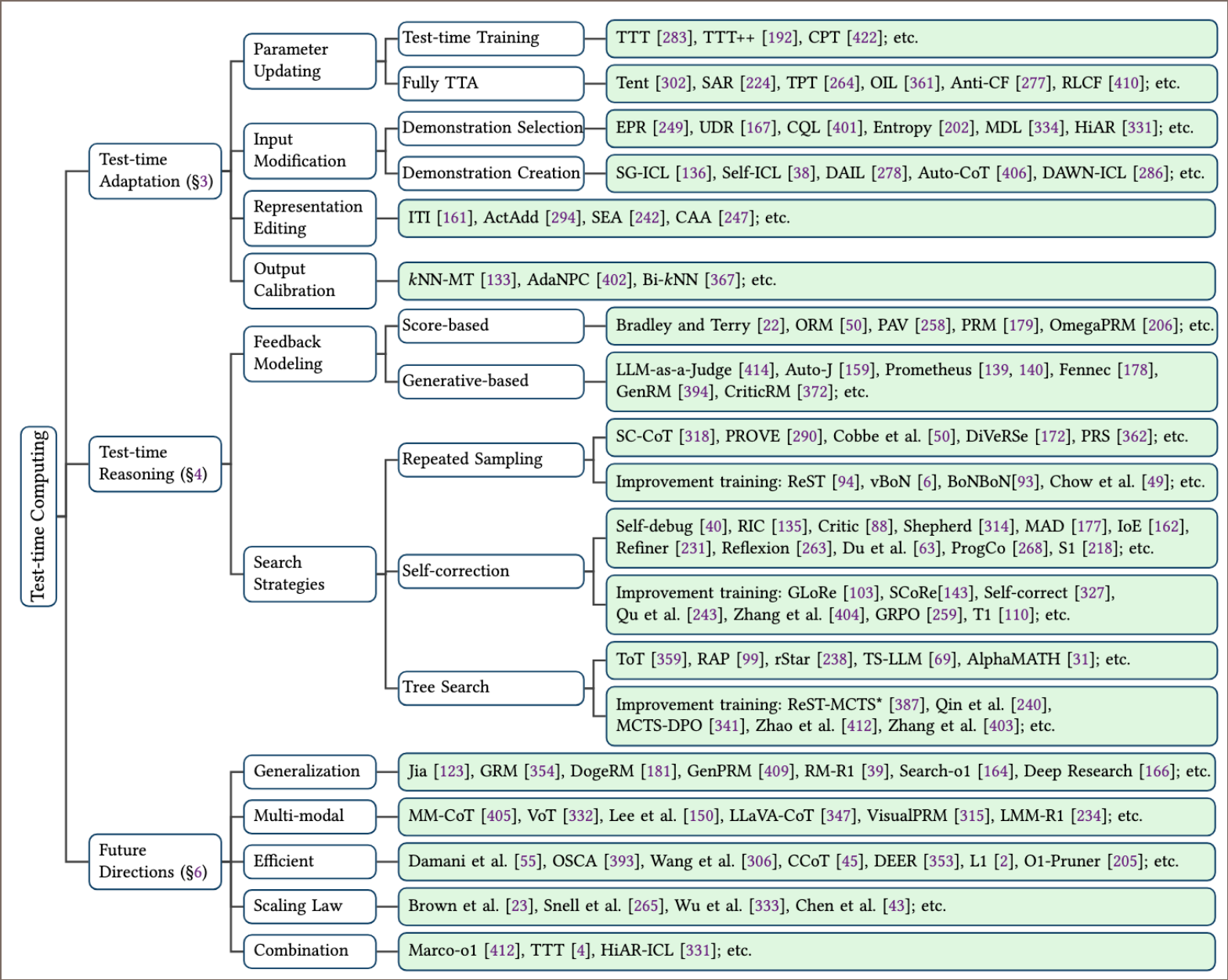
XYZ-of-thoughts & Co

- Tree-of-thought:
linear is limiting
- Graph-of-thought:
acyclicity is limiting
- Program-of-thought:
let there be compute
- Skeleton-of-thought:
sketch and fill-in
- Sampling and voting:
self-consistency wins



from: <https://arxiv.org/abs/2305.10601>

Survey of test-time compute



from: <https://arxiv.org/pdf/2501.02497>

Reinforcement learning with verifiable rewards (RLVR)

- assess accuracy step-by-step already during the reasoning process
- requires an automatic verifier such as unit tests, proof checkers, or compilers
- uses PPO or GRPO with verifiable reward function
- Example of training data:

```
{"id": "ex-002", "input": "Train A leaves at 1pm at 60 km/h, Train B leaves 30 min later at 80 km/h, same track. When does B catch A?",  
  "steps": [  
    {"text": "Define t as hours since 1pm.", "is_correct": true},  
    {"text": "Equate 60t = 80t - 40.", "is_correct": true},  
    {"text": "Solve: t = 1.", "is_correct": false, "error_type": "algebra_mistake",  
  "hint": "Rearrange terms carefully."},  
    {"text": "Correct: 20t=40 => t=2.", "is_correct": true}],  
  "final_answer": {"text": "3:00 PM", "is_correct": true}}
```

Dishes of the Day

- ✓ Instruction tuning
- ✓ Parameter-efficient finetuning
- ✓ Preference alignment
- ✓ Process supervision

